

Quantifying Domain Shift for Adaptive Exploration in Deep Reinforcement Learning

Dylan Forde

Abstract

Reinforcement learning agents often degrade when deployed in environments whose dynamics differ from those observed during training. Standard exploration schedules, such as exponentially decayed ϵ -greedy exploration, do not explicitly react to such domain shifts. This paper presents a compact domain-shift predictor for deep reinforcement learning that estimates the suitability of the current policy under measured changes in environment parameters. The predictor receives the agent state and a scalar domain-shift magnitude, then produces a suitability score used to modulate exploration. We evaluate the approach against a control agent that observes the same domain-shift metric but follows a conventional exploration schedule. Experiments in modified CartPole and MountainCar environments show that adaptive exploration can improve convergence stability and reward under linear, non-linear, and multi-parameter shifts. The results suggest that lightweight shift quantification can be a useful mechanism for making value-based reinforcement learning agents more responsive in non-stationary settings.

1 Introduction

Reinforcement learning (RL) algorithms learn policies by interacting with an environment and maximising cumulative reward [9]. Deep RL extends this framework by using neural networks to approximate value functions or policies, enabling agents to operate in larger state spaces [4]. A persistent obstacle, however, is that many agents are trained under assumptions that the environment dynamics remain stationary or that training variation is sufficient to cover deployment conditions. In practical settings, this assumption is often violated. A robot may experience changes in friction or payload, a controller may face altered physical parameters, and a simulated benchmark may expose the agent to dynamics different from those used during training.

This mismatch is commonly described as domain shift. Existing responses include domain randomisation, transfer learning, and meta-learning [10, 7, 3]. These methods can be effective, but they often depend on broad pre-training distributions, auxiliary tasks, or adaptation procedures that are not directly coupled to the agent’s moment-to-moment perception of the shift. For agents trained online in non-stationary environments, a simpler question is useful: can the agent estimate how suitable its current policy is under the current shift, and use that estimate to adjust exploration?

This work studies that question by augmenting a Deep Q-Network (DQN) agent with a domain-shift predictor. The predictor is deliberately lightweight. It takes the current state and a quantified shift metric as input and outputs a suitability score in $[0, 1]$. Low suitability indicates that the current policy may be poorly matched to the shifted dynamics, so the agent increases exploration by resetting or slowing its ϵ decay. A control agent receives the same domain-shift metric as part of its state input, but does not use the predictor to alter exploration.

The contribution is not a new RL algorithm in the full architectural sense. Instead, it is an adaptive exploration mechanism that can be attached to a value-based agent:

- a scalar domain-shift metric computed from changes in environment parameters;
- a neural predictor that estimates policy suitability from state and shift magnitude;
- an ϵ -greedy adjustment rule that reacts to low predicted suitability;
- an empirical comparison against non-adaptive control agents under several synthetic domain shifts.

The resulting manuscript is condensed from a dissertation-style report into a publication-oriented form. It removes tutorial background, implementation listings, front matter, and speculative application sections, focusing instead on the core method, experimental design, and observed results.

2 Related Work

The exploration-exploitation trade-off is central to RL. In ϵ -greedy exploration, an agent selects a random action with probability ϵ and otherwise chooses the action with the highest estimated value. A typical schedule decays ϵ as training progresses:

$$\epsilon_t = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \exp\left(-\frac{t}{\tau_\epsilon}\right), \quad (1)$$

where τ_ϵ controls the rate of decay. Such schedules are simple and widely used, but they are time dependent rather than shift dependent. Once exploration has decayed, the agent may be slow to recover when the environment changes.

Curiosity-driven and intrinsically motivated exploration methods address sparse rewards by encouraging agents to visit novel or unpredictable states. The Intrinsic Curiosity Module (ICM), for example, uses prediction error in learned dynamics as an intrinsic reward [5]. Reactive Exploration extends this line of work by considering non-stationarity and adapting exploration when the environment changes [8]. The present work is closest in spirit to that adaptive view, but uses an explicit scalar domain-shift metric and a suitability predictor rather than relying only on novelty or surprise.

Domain randomisation trains agents across many sampled environment configurations so that the learned policy generalises to unseen variants [10]. Transfer learning and progressive networks reuse knowledge from previous domains [7, 6], while meta-learning methods such as MAML aim to make models rapidly adaptable after a small number of gradient updates [3]. These approaches are complementary to the method studied here. The proposed predictor does not replace broad training distributions or transfer mechanisms; it provides a direct signal for when the current policy may need more exploration.

3 Method

3.1 Shifted MDP Formulation

The standard DQN setting assumes a fixed Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where the transition kernel $P(s_{t+1} | s_t, a_t)$ and reward function $R(s_t, a_t)$ are stationary. The experiments in this work instead use a family of related MDPs indexed by an environment parameter vector p_t :

$$\mathcal{M}_t = (\mathcal{S}, \mathcal{A}, P_{p_t}, R_{p_t}, \gamma). \quad (2)$$

When $p_t = p_0$, the environment is in its nominal domain. When $p_t \neq p_0$, the transition and reward dynamics may differ from those under which the policy was previously effective. The central mathematical change is therefore that the agent does not act only on s_t . It acts on an augmented observation

$$z_t = [s_t, d_t], \quad (3)$$

where d_t is a scalar quantification of the current domain shift. The value function learned by the DQN is consequently

$$Q_\theta(z_t, a_t) = Q_\theta([s_t, d_t], a_t), \quad (4)$$

rather than the standard $Q_\theta(s_t, a_t)$. This makes the control and DSP agents both shift-aware at the value-function level. The difference between the two variants is that only DSP uses the additional suitability estimate to alter the exploration process.

3.2 Base Agent

The standard one-step DQN target for a stationary MDP is

$$y_t^{\text{std}} = r_t + \gamma(1 - b_t) \max_{a'} Q_{\theta^-}(s_{t+1}, a'), \quad (5)$$

where b_t is a terminal indicator and Q_{θ^-} is the target network. In the shifted setting, the target is evaluated on augmented observations:

$$y_t^{\text{shift}} = r_t + \gamma(1 - b_t) \max_{a'} Q_{\theta^-}(z_{t+1}, a') = r_t + \gamma(1 - b_t) \max_{a'} Q_{\theta^-}([s_{t+1}, d_{t+1}], a'). \quad (6)$$

The corresponding temporal-difference error is

$$\delta_t^{\text{shift}} = y_t^{\text{shift}} - Q_\theta(z_t, a_t). \quad (7)$$

The policy network is trained by minimising a robust temporal-difference loss, implemented as the Huber loss:

$$\mathcal{L}_{\text{DQN}}(\theta) = \frac{1}{N} \sum_{i=1}^N \text{Huber}(y_i^{\text{shift}} - Q_\theta(z_i, a_i)). \quad (8)$$

Equations 6–8 are the important departure from the standard DQN equations: the target and loss are not computed over the state alone, but over a state-shift pair. The target network therefore learns how the value of an action changes as the environment moves away from its nominal parameters.

3.3 Domain-Shift Quantification

For an environment parameter vector p_t and its nominal value p_0 , the domain-shift magnitude is computed as

$$d_t = \sum_{i=1}^k |p_{t,i} - p_{0,i}|. \quad (9)$$

For the single-parameter case, Equation 9 reduces to $d_t = |p_t - p_0|$. For the multi-parameter case, such as simultaneous mass and friction changes, it becomes an additive deviation:

$$d_t = |m_t - m_0| + |\mu_t - \mu_0|, \quad (10)$$

where m_t denotes cart mass and μ_t denotes the shifted friction or force term. This metric is intentionally simple. In CartPole, shifted parameters include pole length, cart mass, and force or friction terms depending on the experiment. In MountainCar, the force parameter is shifted. More expressive distances, such as normalised Euclidean distance or trajectory-based measures, are left for future work.

3.4 Suitability Predictor

The domain-shift predictor is a feed-forward neural network f_ϕ that maps the current state and shift magnitude to a suitability score:

$$u_t = f_\phi([s_t, d_t]), \quad u_t \in [0, 1]. \quad (11)$$

A high score indicates that the current policy is expected to remain suitable under the observed shift, while a low score indicates that the agent should explore more aggressively. The predictor introduces a second learning signal beside the DQN temporal-difference loss. Its target is a binary proxy for policy suitability:

$$y_t^{\text{DSP}} = \begin{cases} 1 & \text{if } b_t = 0, \\ 0 & \text{if } b_t = 1, \end{cases} \quad (12)$$

where b_t is the terminal indicator. This target is coarse, because a transition can be non-terminal even when the policy is suboptimal. It is nevertheless useful because it can be generated online from the same interaction stream as the RL update, without requiring an external domain classifier or manual annotation.

The predictor is trained with binary cross-entropy:

$$\mathcal{L}_{\text{DSP}}(\phi) = -\frac{1}{N} \sum_{i=1}^N y_i^{\text{DSP}} \log u_i + (1 - y_i^{\text{DSP}}) \log(1 - u_i). \quad (13)$$

The predictor and DQN are therefore optimised with two different losses: $\mathcal{L}_{\text{DQN}}$ updates action values, while $\mathcal{L}_{\text{DSP}}$ updates the estimate of whether the current state-shift pair is suitable for the current policy. During the earliest training phase, random suitability values can be injected to avoid a degenerate predictor that returns a constant value before the agent has experienced enough variation. This warm-up is a practical stabilisation measure rather than a core theoretical component.

Table 1: Hyperparameter search space used for model comparison.

Parameter	Search range
Learning rate	10^{-5} to 10^{-2}
Epsilon decay	100 to 2000
Batch size	$\{32, 64, 128, 256\}$
Discount factor γ	0.8 to 0.9999

3.5 Adaptive Exploration Rule

The control agent follows the standard decayed ϵ schedule in Equation 1. In the standard schedule, the decay counter is the global time step t . DSP replaces this with a local counter c_t that can be reset when the suitability estimate becomes too low:

$$\epsilon_t^{\text{DSP}} = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \exp\left(-\frac{c_t}{\tau_\epsilon}\right), \quad (14)$$

$$c_{t+1} = \begin{cases} 0 & \text{if } u_t < \eta, \\ c_t + 1 & \text{otherwise.} \end{cases} \quad (15)$$

The induced behaviour policy is

$$\pi_t(a \mid z_t, u_t) = \begin{cases} 1 - \epsilon_t^{\text{DSP}} + \frac{\epsilon_t^{\text{DSP}}}{|\mathcal{A}|} & \text{if } a = \arg \max_{a'} Q_\theta(z_t, a'), \\ \frac{\epsilon_t^{\text{DSP}}}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \quad (16)$$

This differs from the standard ϵ -greedy policy because ϵ_t^{DSP} is not a function of time alone. It is a function of the suitability signal through the reset rule in Equation 15. The practical effect is to make the exploration schedule responsive to the detected environmental context. Rather than relying only on elapsed training time, the agent can reopen exploration when its learned policy appears poorly matched to the current domain.

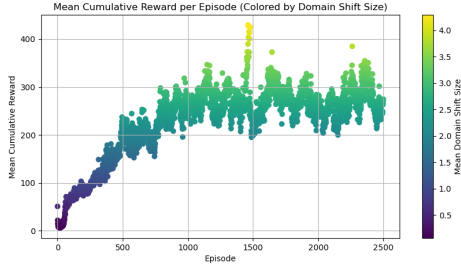
4 Experimental Setup

Experiments were conducted in modified Gym benchmark environments. The primary environment was a custom CartPole variant in which selected physical parameters changed during training. Additional MountainCar experiments tested whether the same mechanism behaved reasonably in a different control task. The agent with the domain-shift predictor is referred to as DSP. The control condition uses the same DQN structure and observes the same scalar domain-shift metric, but does not use the suitability predictor to change exploration.

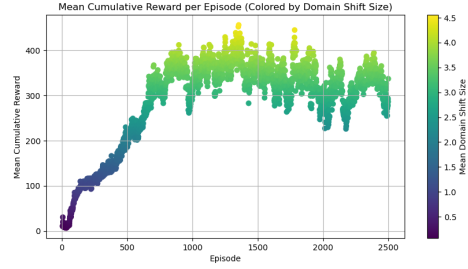
The comparison focuses on four settings:

1. linear CartPole shifts, where pole length changes gradually;
2. non-linear or random CartPole shifts, where changes are more volatile;
3. simultaneous CartPole shifts in friction and mass;
4. MountainCar force shifts.

Hyperparameters were tuned with Optuna. The search ranges are summarised in Table 1. Trials were evaluated by mean cumulative reward over recent episodes, and the analysis compares training curves and optimisation traces between DSP and non-DSP variants.

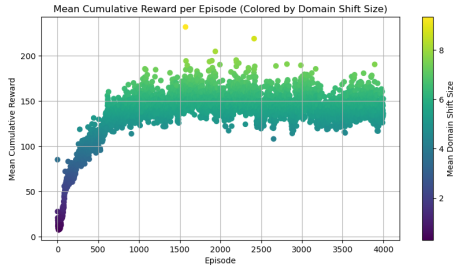


(a) Control

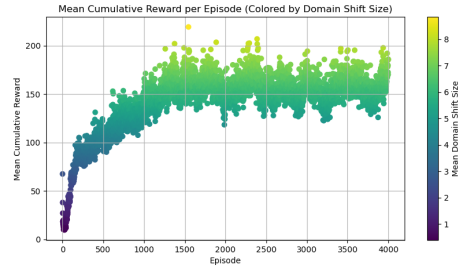


(b) DSP

Figure 1: Training under linear CartPole shifts. The DSP condition shows a higher and more stable reward band.



(a) Control



(b) DSP

Figure 2: Training under random CartPole shifts. Adaptive exploration improves the concentration of successful trials.

5 Results

5.1 Linear CartPole Shifts

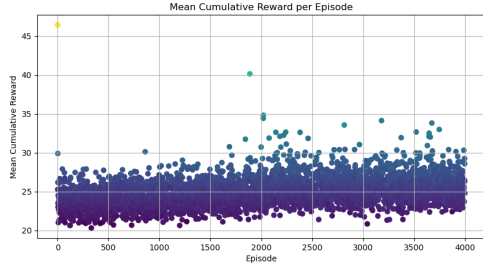
Figure 1 compares CartPole training with linear changes in pole length. The control condition converges mostly in the 200–300 mean reward range, with occasional higher outliers. The DSP condition is more concentrated around the 300–400 range and shows lower variability during convergence. This is notable because a smooth linear shift is the case where a predictor might be expected to offer the least benefit. The observed improvement suggests that even gradual non-stationarity can benefit from shift-aware exploration.

5.2 Volatile CartPole Shifts

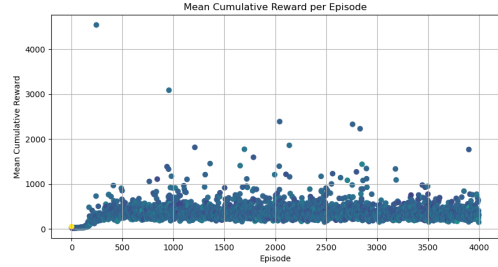
Figure 2 shows training under more volatile CartPole shifts. The control model stabilises around lower mean reward values, while DSP runs more often reach or exceed a 200 reward threshold. The key effect is not merely a higher maximum but a shift in the distribution of successful trials. DSP produces more consistent high-reward behaviour across the training runs shown.

5.3 Multiple Simultaneous Shifts

The strongest contrast appears when more than one parameter changes at the same time. Figure 3 compares models under simultaneous friction and mass changes. The DSP condition maintains substantially higher cumulative reward throughout training, while the control condition remains low and unstable. This supports the central hypothesis: when environmental changes are large enough that a fixed exploration schedule is poorly matched to the task, an explicit suitability signal can help the agent re-enter exploration and adapt more effectively.

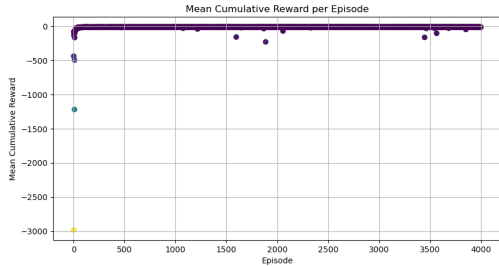


(a) Control

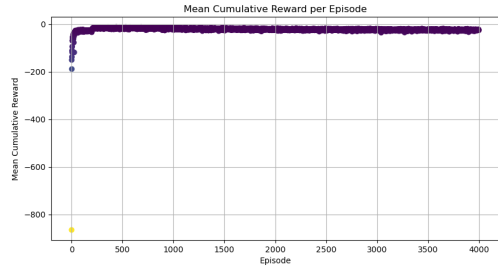


(b) DSP

Figure 3: CartPole with simultaneous friction and mass shifts. The DSP condition is markedly more robust.



(a) Control



(b) DSP

Figure 4: MountainCar with force shifts. DSP mainly improves stability and reduces severe early failures.

5.4 MountainCar Force Shifts

Figure 4 evaluates force shifts in MountainCar. The DSP and control conditions both struggle with the task, but DSP avoids the worst early reward collapses and shows more stable behaviour. This result is more modest than the CartPole multi-shift result, indicating that the method is not uniformly sufficient across all environments. Still, the reduced instability suggests that shift-aware exploration can be useful even when it does not fully solve the task.

6 Discussion

The experiments indicate that a small domain-shift predictor can improve adaptive exploration in non-stationary RL benchmarks. The clearest gains occur when shifts are volatile or multi-parameter. In these settings, the control agent’s decayed exploration schedule appears too rigid: once ϵ has decreased, the agent has limited ability to gather new information after the environment changes. DSP addresses this by linking exploration to the predicted suitability of the current policy.

The method has three practical advantages. First, it is lightweight: the predictor is a small neural network trained online from the same interaction data as the policy. Second, the control comparison is strict in one important respect: both agents observe the domain-shift metric, so the improvement is not simply caused by adding shift information to the DQN input. Third, the mechanism is modular. The predictor influences exploration but does not require changing the DQN objective itself.

There are also important limitations. The scalar shift metric is hand-designed and environment-specific. The binary suitability target is coarse, because episode termination is only an indirect label for policy mismatch. Some predictor runs can become insensitive and output near-constant suitability values, motivating the early random warm-up described above. The evaluation is also limited to classic-control benchmarks and should be extended to more complex continuous-control environments before making stronger claims about real-world deployment.

Future work should replace the absolute-difference metric with normalised multidimensional distances or trajectory-based measures. For example, dynamic time warping could compare state trajectories before and after shifts, while representation-learning methods could quantify movement in a learned state manifold [2, 1, 11]. The same predictor concept should also be evaluated with actor-critic algorithms,

where exploration can be adjusted through policy entropy, action noise, or uncertainty-aware objectives rather than only ϵ -greedy action selection.

7 Conclusion

This paper presented a publication-oriented summary of a domain-shift predictor for adaptive exploration in deep reinforcement learning. The approach quantifies changes in environment parameters, predicts the suitability of the current policy under those changes, and uses the prediction to adjust exploration. Experiments in modified CartPole and MountainCar environments show that the mechanism can improve convergence stability and reward, especially under volatile and simultaneous domain shifts. The results support the broader idea that RL agents should not treat exploration as a purely time-decayed quantity; instead, exploration can be tied to the agent’s estimate of whether its current policy remains suitable for the environment it is actually facing.

References

- [1] Taha Belkhouja, Yan Yan, and Janardhan Rao Doppa. Dynamic time warping based adversarial framework for time-series domain. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(6):7353–7366, 2023.
- [2] Karl Bringmann, Nick Fischer, Ivor van der Hoog, Evangelos Kipouridis, Tomasz Kociumaka, and Eva Rotenberg. Dynamic dynamic time warping, 2023.
- [3] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks, 2017.
- [4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [5] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the International Conference on Machine Learning*, 2017.
- [6] Andrei A. Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell. Policy distillation, 2016.
- [7] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks, 2022.
- [8] Christian Steinparz, Thomas Schmied, Fabian Paischer, Marius-Constantin Dinu, Vihang Patil, Angela Bitto-Nemling, Hamid Eghbal-zadeh, and Sepp Hochreiter. Reactive exploration to cope with non-stationarity in lifelong reinforcement learning, 2022.
- [9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018.
- [10] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world, 2017.
- [11] Zhuangdi Zhu, Kaixiang Lin, Anil K. Jain, and Jiayu Zhou. Transfer learning in deep reinforcement learning: A survey, 2023.